TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN
HƯỚNG DẪN THỰC HÀNH (HANDS-ON LAB)

MÔN HỌC: Các công nghệ lập trình hiện đại (841072)

ĐỀ TÀI: Nghiên cứu Spring Boot – xây dựng hệ thống chat real-time bằng WebSocket

Nhóm: 24 – Lớp: DCT1221 – Nhóm học phần: 01

SINH VIÊN THỰC HIỆN:

HỌ VÀ TÊN	MSSV
Huỳnh Thanh Hải	3123410091
Lê Quốc Cường	3123410040
Nguyễn Phú Tài	3121410435

Giảng viên: Phạm Thi Vương

TP. HỒ CHÍ MINH, THÁNG 11 NĂM 2026

HƯỚNG DẪN THỰC HÀNH TỪNG BƯỚC (HANDS-ON LAB)

Xây dựng Hệ thống Nhắn tin Thời gian thực NextChat bằng Spring Boot 4.1.1, WebSocket/STOMP, JWT và PostgreSQL 16

MỤC LỤC

1. Giới thiệu & Mục tiêu bài Hands-on Lab

2. Chuẩn bị Môi trường & Công cụ (Prerequisites)

3. Cách 1: Khởi chạy Nhanh Dự án từ Kho mã Nguồn (Quick Start trong 5 phút)

4. Cách 2: Thực hành Xây dựng Lại Toàn bộ Dự án Từng Bước (Step-by-Step)

- Bước 1: Khởi tạo Dự án Maven, Docker Compose & Cấu hình Ngoài
- Bước 2: Thiết kế Lược đồ CSDL PostgreSQL 16, Flyway & Hàm PL/pgSQL
- Bước 3: Xây dựng Tầng Thực thể JPA (Entities) & Repositories chống N+1
- Bước 4: Xây dựng Khung Phản hồi Chuẩn (`ApiResponse`) & Xử lý Lỗi Tập trung
- Bước 5: Xây dựng Bảo mật Spring Security, Stateless JWT & Refresh Token Rotation
- Bước 6: Xây dựng REST API Người dùng & Phòng Chat (`User`, `Room`)
- Bước 7: Xây dựng Kênh WebSocket / STOMP & Bảo mật Channel Interceptor
- Bước 8: Xây dựng Giao diện Client Web Thử nghiệm (`index.html` & `app.js`)

5. Thực hành Kiểm thử Tự động, Kiểm chứng Thực nghiệm & Demo End-to-End

6. Bảng Xử lý Sự cố Thường gặp (Troubleshooting Guide)

1. Giới thiệu & Mục tiêu bài Hands-on Lab

Tài liệu **Hands-on Lab** này được biên soạn nhằm hướng dẫn người đọc tự tay thiết lập, xây dựng lại từ đầu, vận hành và kiểm chứng toàn bộ hệ thống **NextChat** — ứng dụng nhắn tin thời gian thực sử dụng **Spring Boot 4.1.1 (Java 21)**, **Spring Security (Stateless JWT + Refresh Token Rotation)**, **Spring Data JPA**, **Spring WebSocket/STOMP**, **Flyway** và **PostgreSQL 16**.

Sau khi hoàn thành bài thực hành này, người đọc sẽ nắm vững và tự thực hiện được:

1. Tổ chức dự án Spring Boot theo kiến trúc **Package-by-Feature + Common** (`auth`, `user`, `room`, `common`).

2. Cấu hình PostgreSQL 16 bằng Docker Compose kết hợp **Flyway Migration** (`V1`, `V2`) và chế độ kiểm chứng lược đồ nghiêm ngặt `spring.jpa.hibernate.ddl-auto: validate`.

3. Giải quyết bài toán **Race Condition** khi hai người dùng tạo phòng chat 1-1 đồng thời bằng hàm **PL/pgSQL** `get_or_create_direct_room` và khóa tư vấn mức giao dịch `pg_advisory_xact_lock`.

4. Tối ưu hóa truy vấn danh sách phòng bằng `@EntityGraph` (chống lỗi **N+1 query** khi tắt `open-in-view: false`) và phân trang lịch sử tin nhắn bằng hàm PL/pgSQL `get_room_messages`.

5. Triển khai cơ chế xác thực **JWT không trạng thái (Stateless)** kết hợp **Refresh Token xoay vòng (Rotation)** lưu trữ dưới dạng băm **SHA-256 Base64URL**.

6. Xây dựng kênh truyền thông thời gian thực **WebSocket/STOMP** có kiểm soát truy cập tại tầng `ChannelInterceptor` (`CONNECT`, `SUBSCRIBE`, `SEND`) và cơ chế **lưu CSDL trước — phát broadcast sau**.

7. Tích hợp giao diện **Client Web thử nghiệm** (`index.html`, `app.js`) và chạy bộ kiểm thử tự động

JUnit 5 + Testcontainers cùng các kịch bản kiểm chứng thực nghiệm.

2. Chuẩn bị Môi trường & Công cụ (Prerequisites)

Trước khi bắt đầu, hãy đảm bảo máy tính của bạn (Linux, macOS hoặc Windows với WSL2) đã cài đặt các công cụ sau:

Công cụ	Phiên bản yêu cầu	Lệnh kiểm tra trên Terminal	Ghi chú
JDK (Java)	**Java 21 LTS** (OpenJDK 21)	`java -version`	Bắt buộc tương thích với Spring Boot 4.1.1
Docker & Compose	Docker Engine 24+ & Compose v2	`docker info && docker compose version`	Dùng để chạy PostgreSQL 16 và Testcontainers
Git	Git 2.30+	`git --version`	Quản lý mã nguồn và clone repository
Maven Wrapper	Đi kèm trong dự án (`./mvnw`)	`./mvnw -version`	Không cần cài đặt Maven toàn cục
cURL / Trình duyệt	Chrome / Firefox / Edge + `curl`	`curl --version`	Kiểm thử REST API và giao diện WebSocket
Python 3 *(Tuỳ chọn)*	Python 3.10+	`python3 --version`	Phục vụ chạy kịch bản E2E Smoke & SQL Lab

3. Cách 1: Khởi chạy Nhanh Dự án từ Kho mã Nguồn (Quick Start trong 5 phút)

Nếu bạn muốn tải mã nguồn hoàn chỉnh của Nhóm 24 về máy để chạy thử và quan sát hệ thống ngay lập tức trước khi đi vào chi tiết từng file mã nguồn, hãy thực hiện đúng **6 bước** dưới đây:

Bước 3.1. Clone kho mã nguồn từ GitHub

```
git clone https://github.com/QuoocsCuongwf/springboot-websocket-chat.git
cd springboot-websocket-chat/nextchat
```

Bước 3.2. Tạo file cấu hình biến môi trường `.env`

Sao chép file mẫu `.env.example` sang `.env` và cấu hình khóa bí mật JWT dài tối thiểu 32 bytes:

```
cp .env.example .env
```

Nội dung chuẩn của file `nextchat/.env`:

```
POSTGRES_DB=messenger_db
POSTGRES_USER=postgres
POSTGRES_PASSWORD=change-this-database-password
POSTGRES_PORT=5432
NEXTCHAT_JWT_SECRET=nextchat-super-secret-jwt-key-at-least-32-bytes-long-2026
```

Bước 3.3. Khởi động container PostgreSQL 16 bằng Docker Compose

```
docker compose --env-file .env up -d
```

Kiểm tra trạng thái container đến khi cột `STATUS` hiển thị `(healthy)`:

```
docker compose ps
```

Bước 3.4. Nạp biến môi trường và khởi chạy ứng dụng Spring Boot

```
set -a && . ./.env && set +a
./mvnw spring-boot:run
```

Khi ứng dụng khởi động thành công, bạn sẽ thấy trên console:

• Flyway tự động thực thi `V1__messenger_schema.sql` và `V2__lock_direct_room_creation.sql`.

• Hibernate kiểm chứng khớp toàn bộ lược đồ ở chế độ `ddl-auto: validate`.

• `SeedDataConfig` khởi tạo sẵn tài khoản thử nghiệm `testuser` (mật khẩu mặc định ở profile `dev`: `ChangeMe123!`).

• Dòng thông báo: `Started NextchatApplication in ... seconds`.

Bước 3.5. Trải nghiệm ngay trên Giao diện Web

1. Mở trình duyệt ở cửa sổ thường tại địa chỉ: `http://localhost:8080`

2. Mở thêm một cửa sổ ẩn danh (Incognito) tại địa chỉ: `http://localhost:8080`

3. Đăng ký 2 tài khoản (ví dụ `alice` và `bob`), đăng nhập ở 2 cửa sổ, tìm kiếm nhau ở ô **Tìm người dùng**, bấm tạo phòng chat 1-1 và nhắn tin qua lại thời gian thực.

4. Cách 2: Thực hành Xây dựng Lại Toàn bộ Dự án Từng Bước (Step-by-Step)

Phần này hướng dẫn bạn tự tay xây dựng lại toàn bộ dự án NextChat từ cấu trúc thư mục rỗng đến khi hoàn chỉnh tất cả các tầng.

Tổng quan Cấu trúc Thư mục Dự án

```
springboot-websocket-chat/
├── database.yml # Cấu hình HikariCP, JPA validate, tắt OSIV
├── README.md
└── nextchat/
    ├── pom.xml # Khai báo phụ thuộc Spring Boot 4.1.1 & Java 21
    ├── docker-compose.yml # Định nghĩa container postgres:16-alpine
    ├── .env.example # Mẫu biến môi trường
    └── src/
        ├── main/
        │ ├── java/com/quoocscuongwf/nextchat/
        │ │ ├── NextchatApplication.java
        │ │ ├── common/
        │ │ │ ├── config/ # WebSecurityConfig, WebSocketConfig, PasswordConfig,
SeedDataConfig
        │ │ │ ├── dto/ # ApiResponse<T>
        │ │ │ ├── exception/ # GlobalExceptionHandler
        │ │ │ └── security/ # JwtService, JwtAuthenticationFilter,
JwtStompChannelInterceptor, ...
        │ │ ├── auth/ # AuthController, RefreshToken, RefreshTokenRepository,
RefreshTokenService, dto/
        │ │ ├── user/ # User, Role, UserRepository, RoleRepository, UserController, dto/
        │ │ └── room/ # Conversation, Message, Participant, RoomService, RoomController,
RoomWebSocketController, dto/
        │ └── resources/
        │ ├── application.yaml # Cấu hình trung tâm của Spring Boot
        │ ├── db/migration/
        │ │ ├── V1__messenger_schema.sql # Lược đồ 20 bảng + hàm PL/pgSQL
        │ │ └── V2__lock_direct_room_creation.sql
        │ └── static/
        │ ├── index.html # Giao diện SPA kiểm thử REST & STOMP
        │ └── app.js # Logic xử lý REST, Refresh Token & STOMP client
        └── test/
            └── java/com/quoocscuongwf/nextchat/
                ├── api/AuthAndRoomIntegrationTest.java
                └── security/JwtStompChannelInterceptorTest.java
```

Bước 1: Khởi tạo Dự án Maven, Docker Compose & Cấu hình Ngoài

1.1. Thiết lập file `nextchat/pom.xml`

Tạo file `nextchat/pom.xml` sử dụng `spring-boot-starter-parent` phiên bản `4.1.1` và

`java.version` là `21`. Lưu ý ở thế hệ Spring Boot 4, starter cho Web MVC mang tên `spring-boot-starter-webmvc` và starter cho Flyway là `spring-boot-starter-flyway`:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>4.1.1</version>
        <relativePath/>
    </parent>
    <groupId>com.quoocscuongwf</groupId>
    <artifactId>nextchat</artifactId>
    <version>0.0.1-SNAPSHOT</version>

    <properties>
        <java.version>21</java.version>
        <testcontainers.version>1.21.4</testcontainers.version>
    </properties>

    <dependencies>
        <!-- Spring Boot Core Starters -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-jackson</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-websocket</artifactId>
        </dependency>

        <!-- Database & Migration (PostgreSQL 16 + Flyway) -->
        <dependency>
            <groupId>org.postgresql</groupId>
            <artifactId>postgresql</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-flyway</artifactId>
        </dependency>
        <dependency>
            <groupId>org.flywaydb</groupId>
            <artifactId>flyway-database-postgresql</artifactId>
        </dependency>

        <!-- Thư viện JJWT 0.12.6 -->
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-api</artifactId>
```

```
            <version>0.12.6</version>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-impl</artifactId>
            <version>0.12.6</version>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-jackson</artifactId>
            <version>0.12.6</version>
            <scope>runtime</scope>
        </dependency>

        <!-- Test Dependencies (JUnit 5 + Testcontainers) -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-websocket-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.testcontainers</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>${testcontainers.version}</version>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.testcontainers</groupId>
            <artifactId>postgresql</artifactId>
            <version>${testcontainers.version}</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
</project>
```

1.2. Thiết lập `nextchat/docker-compose.yml`

Tạo file `nextchat/docker-compose.yml` để khởi chạy PostgreSQL 16 kèm `healthcheck`:

```
services:
  postgres:
    image: postgres:16-alpine
    container_name: nextchat-postgres
    restart: unless-stopped
    environment:
      POSTGRES_DB: ${POSTGRES_DB:-nextchat}
      POSTGRES_USER: ${POSTGRES_USER:-nextchat}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-nextchat123}
    ports:
      - "${POSTGRES_PORT:-5432}:5432"
    volumes:
```

```
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-nextchat} -d ${POSTGRES_DB:-
nextchat}"]
      interval: 10s
      timeout: 5s
      retries: 5

volumes:
  postgres_data:
```

1.3. Thiết lập `database.yml` và `nextchat/src/main/resources/application.yaml`

Tạo file `database.yml` ở thư mục gốc của kho mã nhằm quản lý cấu hình HikariCP connection pool, bật `ddl-auto: validate` và tắt `open-in-view: false` (buộc mọi truy vấn lazy phải được giải quyết trong phạm vi `@Transactional` tại tầng Service):

```
# File: database.yml
spring:
  datasource:
    url: ${SPRING_DATASOURCE_URL:jdbc:postgresql://localhost:${POSTGRES_PORT:5432}/$
{POSTGRES_DB:messenger_db}}
    username: ${SPRING_DATASOURCE_USERNAME:${POSTGRES_USER:postgres}}
    password: ${SPRING_DATASOURCE_PASSWORD:${POSTGRES_PASSWORD:change-this-database-
password}}
    driver-class-name: org.postgresql.Driver
    hikari:
      maximum-pool-size: 10
      minimum-idle: 2
      idle-timeout: 30000
      connection-timeout: 20000
      max-lifetime: 1800000
  jpa:
    hibernate:
      ddl-auto: validate
    open-in-view: false
    properties:
      hibernate:
        format_sql: true
        show_sql: false
```

Tạo file `nextchat/src/main/resources/application.yaml`:

```
# File: nextchat/src/main/resources/application.yaml
spring:
  application:
    name: nextchat
  profiles:
    default: dev
  config:
    import:
      - optional:file:../database.yml
      - optional:file:.env
  flyway:
    enabled: true
    locations: classpath:db/migration
    baseline-on-migrate: true
  datasource:
    url: ${SPRING_DATASOURCE_URL:jdbc:postgresql://localhost:${POSTGRES_PORT:5432}/$
{POSTGRES_DB:messenger_db}}
    username: ${SPRING_DATASOURCE_USERNAME:${POSTGRES_USER:postgres}}
    password: ${SPRING_DATASOURCE_PASSWORD:${POSTGRES_PASSWORD:change-this-database-
password}}
  jpa:
    hibernate:
      ddl-auto: validate

app:
  security:
    jwt:
      secret: ${NEXTCHAT_JWT_SECRET:development-only-secret-change-me-before-
production-12345}
```

```
      expiration: ${NEXTCHAT_JWT_EXPIRATION:PT15M}
      refresh-token-expiration: ${NEXTCHAT_JWT_REFRESH_EXPIRATION:P7D}
  seed:
    password: ${NEXTCHAT_SEED_PASSWORD:ChangeMe123!}
```

Bước 2: Thiết kế Lược đồ CSDL PostgreSQL 16, Flyway & Hàm PL/pgSQL

Flyway tự động quét thư mục `src/main/resources/db/migration`. Chúng ta tạo 2 tệp migration theo đúng thứ tự phiên bản:

2.1. Tạo `V1__messenger_schema.sql`

File `V1__messenger_schema.sql` khởi tạo 20 bảng của hệ thống nhắn tin (trong đó 6 bảng cốt lõi được ánh xạ trực tiếp sang JPA Entity: `users`, `roles`, `user_roles`, `refresh_tokens`, `conversations`, `conversation_participants`, `messages`) cùng với trigger cập nhật `updated_at` và 2 hàm PL/pgSQL quan trọng.

Dưới đây là cấu trúc các bảng cốt lõi và hàm phân trang `get_room_messages` trong `V1__messenger_schema.sql`:

```
-- Trigger tự động cập nhật cột updated_at
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Bảng người dùng, vai trò và refresh_tokens
CREATE TABLE users (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(150) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    full_name VARCHAR(150) NOT NULL,
    avatar_url VARCHAR(500),
    bio VARCHAR(255),
    phone_number VARCHAR(20),
    status VARCHAR(10) NOT NULL DEFAULT 'OFFLINE'
                        CHECK (status IN ('ONLINE','OFFLINE','AWAY','BUSY')),
    last_seen_at TIMESTAMP,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    is_deleted BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE roles (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name VARCHAR(30) NOT NULL UNIQUE
);

CREATE TABLE user_roles (
    user_id BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    role_id BIGINT NOT NULL REFERENCES roles(id) ON DELETE CASCADE,
    PRIMARY KEY (user_id, role_id)
);

CREATE TABLE refresh_tokens (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    user_id BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    token VARCHAR(255) NOT NULL UNIQUE,
    expiry_date TIMESTAMP NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Bảng hội thoại (PRIVATE / GROUP), thành viên và tin nhắn
```

```
CREATE TABLE conversations (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    type VARCHAR(10) NOT NULL CHECK (type IN ('PRIVATE','GROUP')),
    name VARCHAR(150),
    avatar_url VARCHAR(500),
    created_by BIGINT NOT NULL REFERENCES users(id),
    last_message_id BIGINT,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE conversation_participants (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    conversation_id BIGINT NOT NULL REFERENCES conversations(id) ON DELETE CASCADE,
    user_id BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    nickname VARCHAR(100),
    role VARCHAR(10) NOT NULL DEFAULT 'MEMBER'
                                CHECK (role IN ('OWNER','ADMIN','MEMBER')),
    is_muted BOOLEAN NOT NULL DEFAULT FALSE,
    is_pinned BOOLEAN NOT NULL DEFAULT FALSE,
    is_archived BOOLEAN NOT NULL DEFAULT FALSE,
    unread_count INT NOT NULL DEFAULT 0,
    last_read_message_id BIGINT,
    joined_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    left_at TIMESTAMP,
    CONSTRAINT uq_conv_participant UNIQUE (conversation_id, user_id)
);

CREATE TABLE messages (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    conversation_id BIGINT NOT NULL REFERENCES conversations(id) ON DELETE CASCADE,
    sender_id BIGINT NOT NULL REFERENCES users(id),
    message_type VARCHAR(10) NOT NULL DEFAULT 'TEXT'
                                  CHECK (message_type IN
('TEXT','IMAGE','VIDEO','AUDIO','FILE','STICKER','SYSTEM')),
    content TEXT,
    reply_to_message_id BIGINT REFERENCES messages(id) ON DELETE SET NULL,
    forwarded_from_id BIGINT REFERENCES messages(id) ON DELETE SET NULL,
    is_edited BOOLEAN NOT NULL DEFAULT FALSE,
    is_deleted BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Hàm PL/pgSQL phân trang lịch sử tin nhắn kèm tổng số bản ghi
CREATE OR REPLACE FUNCTION get_room_messages(
    p_room_id BIGINT,
    p_page INT DEFAULT 0,
    p_size INT DEFAULT 20
)
RETURNS TABLE (
    id BIGINT,
    conversation_id BIGINT,
    sender_id BIGINT,
    sender_username VARCHAR(50),
    sender_full_name VARCHAR(150),
    sender_avatar_url VARCHAR(500),
    message_type VARCHAR(10),
    content TEXT,
    reply_to_message_id BIGINT,
    is_edited BOOLEAN,
    is_deleted BOOLEAN,
    created_at TIMESTAMP,
    updated_at TIMESTAMP,
    total_count BIGINT
) AS $$
DECLARE
    v_offset INT := GREATEST(p_page, 0) * GREATEST(p_size, 1);
    v_total BIGINT;
BEGIN
    SELECT COUNT(*) INTO v_total
```

```
    FROM messages m
    WHERE m.conversation_id = p_room_id
      AND m.is_deleted = FALSE;

    RETURN QUERY
    SELECT
        m.id, m.conversation_id, m.sender_id,
        u.username AS sender_username,
        u.full_name AS sender_full_name,
        u.avatar_url AS sender_avatar_url,
        m.message_type, m.content, m.reply_to_message_id,
        m.is_edited, m.is_deleted, m.created_at, m.updated_at,
        COALESCE(v_total, 0) AS total_count
    FROM messages m
    JOIN users u ON m.sender_id = u.id
    WHERE m.conversation_id = p_room_id
      AND m.is_deleted = FALSE
    ORDER BY m.created_at DESC, m.id DESC
    LIMIT GREATEST(p_size, 1) OFFSET v_offset;
END;
$$ LANGUAGE plpgsql;
```

2.2. Tạo `V2__lock_direct_room_creation.sql` (Chống Race Condition bằng Khóa Advisory)

Khi hai người dùng A và B đồng thời bấm nhắn tin cho nhau ở cùng một mili-giây, mức cô lập mặc định `READ COMMITTED` của PostgreSQL không ngăn được việc cả hai giao dịch cùng kiểm tra thấy "chưa có phòng" và cùng `INSERT` tạo ra 2 phòng chat 1-1 trùng lặp.

Để giải quyết triệt để, tạo file `V2__lock_direct_room_creation.sql` sử dụng khóa tư vấn mức giao dịch `pg_advisory_xact_lock` trên mã băm 64-bit chuẩn hóa thứ tự `LEAST(p_user_a, p_user_b) || ':' || GREATEST(p_user_a, p_user_b)`:

```
CREATE OR REPLACE FUNCTION get_or_create_direct_room(
    p_user_a BIGINT,
    p_user_b BIGINT
)
RETURNS BIGINT AS $$
DECLARE
    v_room_id BIGINT;
BEGIN
    IF p_user_a = p_user_b THEN
        RAISE EXCEPTION 'A private room requires two different users';
    END IF;

    -- Khóa advisory mức giao dịch: cùng một khóa cho cả chiều (A,B) lẫn (B,A)
    PERFORM pg_advisory_xact_lock(
        hashtextextended(
            LEAST(p_user_a, p_user_b)::TEXT || ':' || GREATEST(p_user_a,
p_user_b)::TEXT,
            0
        )
    );

    SELECT cp1.conversation_id INTO v_room_id
    FROM conversation_participants cp1
    JOIN conversation_participants cp2 ON cp1.conversation_id = cp2.conversation_id
    JOIN conversations c ON c.id = cp1.conversation_id
    WHERE c.type = 'PRIVATE'
      AND cp1.user_id = p_user_a
      AND cp2.user_id = p_user_b
      AND cp1.left_at IS NULL
      AND cp2.left_at IS NULL
    ORDER BY c.id
    LIMIT 1;

    IF v_room_id IS NOT NULL THEN
        RETURN v_room_id;
    END IF;
```

```
    INSERT INTO conversations (type, created_by, created_at, updated_at)
    VALUES ('PRIVATE', p_user_a, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP)
    RETURNING id INTO v_room_id;

    INSERT INTO conversation_participants (conversation_id, user_id, role, joined_at)
    VALUES (v_room_id, p_user_a, 'OWNER', CURRENT_TIMESTAMP);

    INSERT INTO conversation_participants (conversation_id, user_id, role, joined_at)
    VALUES (v_room_id, p_user_b, 'MEMBER', CURRENT_TIMESTAMP);

    RETURN v_room_id;
END;
$$ LANGUAGE plpgsql;
```

Bước 3: Xây dựng Tầng Thực thể JPA (Entities) & Repositories chống N+1

1. **Ánh xạ các JPA Entity:** Tạo 6 lớp Entity (`User`, `Role`, `RefreshToken`, `Conversation`, `ConversationParticipant`, `Message`) với `@Id @GeneratedValue(strategy = GenerationType.IDENTITY)` và kiểu thời gian `java.time.Instant` khớp chính xác với các cột trong `V1__messenger_schema.sql`.
2. **Tối ưu truy vấn với `@EntityGraph` trong `ConversationRepository.java`:**

Do chúng ta đã tắt `open-in-view: false`, nếu nạp danh sách phòng rồi duyệt qua `participants` và `lastMessage` sẽ gây ra lỗi `LazyInitializationException` hoặc phát sinh **N+1 câu lệnh SQL**. Chúng ta giải quyết bằng `@EntityGraph`:

```
package com.quoocscuongwf.nextchat.room;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.EntityGraph;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import java.util.List;

public interface ConversationRepository extends JpaRepository<Conversation, Long> {

    @EntityGraph(attributePaths = {
        "participants", "participants.user", "createdBy", "lastMessage",
"lastMessage.sender"
    })
    @Query("""
        SELECT DISTINCT c FROM Conversation c
        JOIN c.participants p
        WHERE p.user.id = :userId AND p.leftAt IS NULL
        ORDER BY c.updatedAt DESC
    """)
    List<Conversation> findRoomsByUserId(@Param("userId") Long userId);

    @Query(value = "SELECT get_or_create_direct_room(:userA, :userB)", nativeQuery =
true)
    Long callGetOrCreateDirectRoom(@Param("userA") Long userA, @Param("userB") Long
userB);
}
```

Bước 4: Xây dựng Khung Phản hồi Chuẩn (`ApiResponse`) & Xử lý Lỗi Tập trung

Tất cả các REST API dưới `/api/v1` đều trả về cùng một cấu trúc phong bì JSON thống nhất thông qua Java `record ApiResponse<T>`:

```
package com.quoocscuongwf.nextchat.common.dto;

import java.time.Instant;

public record ApiResponse<T>(
        boolean success,
        String message,
        T data,
```

```
        Instant timestamp
) {
    public static <T> ApiResponse<T> success(String message, T data) {
        return new ApiResponse<>(true, message, data, Instant.now());
    }

    public static <T> ApiResponse<T> error(String message, T data) {
        return new ApiResponse<>(false, message, data, Instant.now());
    }
}
```

Kết hợp với `@RestControllerAdvice` trong `GlobalExceptionHandler.java` để bắt mọi ngoại lệ (`MethodArgumentNotValidException`, `ResponseStatusException`, `BadCredentialsException`, `DataIntegrityViolationException`) và trả về đúng mã HTTP kèm phong bì `ApiResponse`.

Bước 5: Xây dựng Bảo mật Spring Security, Stateless JWT & Refresh Token Rotation

5.1. Dịch vụ ký & xác minh JWT (`JwtService.java`)

`JwtService` sử dụng thuật toán HMAC-SHA256 (`Keys.hmacShaKeyFor`). Đặc biệt, hàm khởi tạo kiểm tra độ dài khóa tối thiểu 32 bytes và **tự động từ chối khởi động ứng dụng** nếu phát hiện khóa mặc định được dùng ngoài profile `dev` hoặc `test`:

```
if (secret.getBytes(StandardCharsets.UTF_8).length < 32) {
    throw new IllegalArgumentException("JWT secret must be at least 32 bytes");
}
if (DEVELOPMENT_SECRET.equals(secret)
        && !environment.acceptsProfiles(Profiles.of("dev", "test"))) {
    throw new IllegalStateException("NEXTCHAT_JWT_SECRET must be configured outside dev
and test profiles");
}
```

5.2. Xoay vòng Refresh Token băm SHA-256 (`RefreshTokenService.java`)

- Khi người dùng đăng nhập hoặc làm mới token, hệ thống sinh 32 bytes ngẫu nhiên bằng `SecureRandom` (mã hóa Base64URL không padding) để trả về cho client.
- Cơ sở dữ liệu **không lưu token thô** mà chỉ lưu bản băm `SHA-256` ở cột `refresh_tokens.token`.
- Phương thức `rotateRefreshToken(String rawToken)` chạy trong `@Transactional`: tìm bản băm của token cũ, kiểm tra hạn sử dụng và trạng thái tài khoản, **xóa ngay bản ghi token cũ** rồi phát hành cặp token mới. Nếu kẻ xấu đánh cắp và gửi lại refresh token cũ đã qua sử dụng, hệ thống trả về `401 Unauthorized`.

5.3. Cấu hình chuỗi Filter bảo mật (`WebSecurityConfig.java`)

Thiết lập `SessionCreationPolicy.STATELESS`, tắt CSRF (do dùng header `Authorization: Bearer <token>`), mở quyền truy cập cho tài nguyên tĩnh (`/`, `/index.html`, `/app.js`), các endpoint bắt tay WebSocket (`/ws/**`, `/ws-chat/**`) và `/api/v1/auth/**`, đồng thời yêu cầu xác thực cho mọi đường dẫn `/api/**`.

Bước 6: Xây dựng REST API Người dùng & Phòng Chat (`User`, `Room`)

Triển khai đầy đủ 9 REST API endpoints theo hợp đồng chuẩn:

Phương thức	Đường dẫn Endpoint	Mô tả Chức năng
`POST`	`/api/v1/auth/register`	Đăng ký tài khoản mới (`201 Created`)
`POST`	`/api/v1/auth/login`	Đăng nhập, nhận `accessToken` + `refreshToken` xoay vòng

`POST`	`/api/v1/auth/refresh`	Xoay vòng `refreshToken` nhận cặp token mới
`GET`	`/api/v1/users/me`	Lấy thông tin hồ sơ người dùng hiện tại
`GET`	`/api/v1/users/search?q=...`	Tìm kiếm người dùng theo `username` hoặc `fullName`
`GET`	`/api/v1/rooms`	Lấy danh sách phòng chat của người dùng hiện tại
`POST`	`/api/v1/rooms/direct`	Tạo hoặc lấy phòng chat 1-1 nguyên tử (`targetUserId`)
`POST`	`/api/v1/rooms/group`	Tạo phòng chat nhóm (`groupName`, `memberIds`)
`GET`	`/api/v1/rooms/{roomId}/messages`	Phân trang lịch sử tin nhắn (`page`, `size` từ 1 đến 100)

Lưu ý kỹ thuật quan trọng trong `RoomService.getRoomMessages`:

Khi truy vấn một trang nằm ngoài phạm vi dữ liệu (ví dụ `page=5` khi phòng chỉ có 2 tin nhắn), hàm `get_room_messages` trả về 0 dòng nên không thể đọc cột `total_count` từ dòng kết quả. Do đó, `RoomService` cài đặt nhánh dự phòng gọi `messageRepository.countByConversationIdAndDeletedFalse(roomId)` khi `rows.isEmpty()` để luôn trả về đúng `totalElements` cho client.

Bước 7: Xây dựng Kênh WebSocket / STOMP & Bảo mật Channel Interceptor

7.1. Cấu hình Message Broker (`WebSocketConfig.java`)

- Đăng ký 2 endpoint bắt tay: `/ws` (WebSocket thuần phục vụ kiểm thử tự động) và `/ws-chat` (kích hoạt `.withSockJS()` phục vụ trình duyệt web).
- Tiền tố đích ứng dụng: `/app` (ánh xạ vào `@MessageMapping`).
- Tiền tố đích Simple Broker: `/topic` và `/queue`.
- Gắn `JwtStompChannelInterceptor` vào `configureClientInboundChannel`.

7.2. Kiểm soát truy cập STOMP (`JwtStompChannelInterceptor.java`)

1. **Tại khung `CONNECT`:** Đọc JWT từ native header `Authorization: Bearer <token>`, xác minh chữ ký và hạn dùng qua `JwtService`, nạp `UserDetails`, gán `Authentication` vào accessor và lưu `stompUsername` vào `sessionAttributes`.

2. **Tại các khung tiếp theo (`SUBSCRIBE`, `SEND`):** Khôi phục `Authentication` cho khung truyền từ `sessionAttributes` để `@MessageMapping` luôn nhận được `Principal` hợp lệ.

3. **Tại khung `SUBSCRIBE` tới `/topic/room/{roomId}`:** Kiểm tra người dùng có phải là thành viên đang hoạt động (`left_at IS NULL`) của phòng hay không; nếu là người ngoài phòng thì lập tức ném `IllegalArgumentException` từ chối đăng ký.

4. Tăng cường bảo mật (Bản vá khuyến nghị trước bảo vệ): Trong phương thức `preSend`, kiểm tra chặn các đích `SUBSCRIBE` mang ký tự đại diện (`/topic/room/*`) và chặn khung `SEND` gửi trực tiếp vào `/topic/...` (bắt buộc mọi khung `SEND` của client phải đi qua `/app/room/{roomId}`).

7.3. Xử lý gửi tin nhắn (`RoomWebSocketController.java`)

```
@MessageMapping("/room/{roomId}")
public void sendMessage(
        @DestinationVariable Long roomId,
        @Valid @Payload SendMessageRequest request,
        Principal principal) {
    if (principal == null) {
        throw new ResponseStatusException(HttpStatus.UNAUTHORIZED, "Authentication
required");
    }
    Long userId = userRepository.findByUsername(principal.getName())
            .filter(user -> user.isActive() && !user.isDeleted())
            .map(User::getId)
            .orElseThrow(() -> new ResponseStatusException(HttpStatus.UNAUTHORIZED,
"Authentication required"));
    // Lưu vào PostgreSQL và cập nhật last_message_id trong giao dịch trước khi phát
broadcast
    MessageDTO message = roomService.sendMessage(
            userId, roomId, request.content(), request.messageType(),
request.replyToMessageId());
    messagingTemplate.convertAndSend("/topic/room/" + roomId, message);
}
```

Bước 8: Xây dựng Giao diện Client Web Thử nghiệm (`index.html` & `app.js`)

Đặt 2 tệp `index.html` và `app.js` tại `nextchat/src/main/resources/static/` để phục vụ trực tiếp trên cổng `8080`:

- Hàm `apiFetch(path, options, retry = true)` tự động đính kèm `Authorization: Bearer <accessToken>`. Nếu nhận mã `401`, hàm tự động gọi `POST /api/v1/auth/refresh` để xoay vòng token, lưu cặp token mới vào `localStorage`, kết nối lại STOMP và thử lại request ban đầu với `retry = false`.
- Khi người dùng gửi tin nhắn qua `stompClient.send("/app/room/" + activeRoom.id, {}, JSON.stringify(payload))`, giao diện **không tự vẽ tin nhắn cục bộ** mà chờ nhận khung `MESSAGE` từ `/topic/room/{roomId}` do server phát về (kèm `id` và `createdAt` chính thức từ PostgreSQL).

5. Thực hành Kiểm thử Tự động, Kiểm chứng Thực nghiệm & Demo End-to-End

5.1. Chạy Bộ Kiểm thử Tự động JUnit 5 & Testcontainers

Đảm bảo Docker đang chạy, sau đó đứng tại thư mục `nextchat/` và thực thi:

```
cd nextchat
./mvnw clean verify
```

- **Testcontainers** sẽ tự động khởi tạo một container `postgres:16-alpine` cô lập, chạy Flyway migrations (`V1`, `V2`) và thực thi toàn bộ các bài kiểm thử đơn vị (`JwtStompChannelInterceptorTest`) cùng kiểm thử tích hợp (`AuthAndRoomIntegrationTest`), bao gồm cả bài kiểm thử **2 luồng đồng thời tạo phòng 1-1 (`CountDownLatch`)** và kiểm thử gửi/nhận tin nhắn qua STOMP WebSocket thật.

5.2. Kiểm thử Toàn bộ Luồng REST API bằng `curl` trên Terminal

Khi ứng dụng đang chạy ở `http://localhost:8080`, mở một cửa sổ Terminal mới và thực hành tuần tự các lệnh sau:

(1) Đăng ký 2 tài khoản `user_a` và `user_b`

```
curl -s -X POST http://localhost:8080/api/v1/auth/register \
  -H "Content-Type: application/json" \
  -d
'{"username":"user_a","email":"user_a@example.com","password":"Password123!","fullName":
"Nguyen Van A"}'
```

```
curl -s -X POST http://localhost:8080/api/v1/auth/register \
  -H "Content-Type: application/json" \
  -d
'{"username":"user_b","email":"user_b@example.com","password":"Password123!","fullName":
"Tran Thi B"}'
```

(2) Đăng nhập tài khoản `user_a` và trích xuất `ACCESS_TOKEN`, `REFRESH_TOKEN`

```
LOGIN_RES=$(curl -s -X POST http://localhost:8080/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"user_a","password":"Password123!"}')
echo "$LOGIN_RES"
```

(3) Kiểm chứng Tạo phòng 1-1 Nguyên tử (`POST /api/v1/rooms/direct`)

Thay `<ACCESS_TOKEN>` và `<USER_B_ID>` bằng giá trị nhận được ở trên:

```
curl -s -X POST http://localhost:8080/api/v1/rooms/direct \
  -H "Authorization: Bearer <ACCESS_TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"targetUserId": <USER_B_ID>}'
```

Gọi lại lệnh trên nhiều lần hoặc gọi ngược chiều từ `user_b` sang `user_a`: hệ thống luôn trả về **cùng một `id` phòng duy nhất** nhờ hàm `get_or_create_direct_room` và khóa `pg_advisory_xact_lock`.

(4) Kiểm chứng Xoay vòng Refresh Token (`POST /api/v1/auth/refresh`)

```
curl -s -X POST http://localhost:8080/api/v1/auth/refresh \
  -H "Content-Type: application/json" \
  -d '{"refreshToken": "<REFRESH_TOKEN_CU>"}'
```

- Lần gọi thứ nhất trả về `200 OK` kèm cặp `accessToken` và `refreshToken` mới.
- Thử gọi lại đúng lệnh trên một lần nữa với `<REFRESH_TOKEN_CU>`: hệ thống lập tức trả về `401 Unauthorized` (`"Invalid or expired refresh token"`), chứng minh token cũ đã bị thu hồi ngay sau lần sử dụng đầu tiên.

5.3. Kiểm chứng Trực tiếp trong Cơ sở Dữ liệu PostgreSQL

Bạn có thể mở phiên `psql` trực tiếp vào container `nextchat-postgres` để kiểm tra dữ liệu đã lưu:

```
docker exec -it nextchat-postgres psql -U postgres -d messenger_db
```

Thực thi các câu truy vấn kiểm chứng:

```
-- 1. Kiểm tra lịch sử migration Flyway
SELECT installed_rank, version, description, success FROM flyway_schema_history;

-- 2. Kiểm tra Refresh Token chỉ lưu dưới dạng băm SHA-256 (43 ký tự Base64URL)
SELECT id, user_id, token, LENGTH(token) AS hash_len, expiry_date FROM refresh_tokens;

-- 3. Kiểm tra danh sách phòng chat và tin nhắn cuối cùng
SELECT id, type, name, created_by, last_message_id, updated_at FROM conversations;

-- 4. Kiểm tra hàm phân trang PL/pgSQL get_room_messages
SELECT id, sender_username, content, created_at, total_count FROM get_room_messages(1,
0, 10);
```

6. Bảng Xử lý Sự cố Thường gặp (Troubleshooting Guide)

Triệu chứng / Thông báo lỗi	Nguyên nhân gốc rễ	Cách khắc phục triệt để
`IllegalArgumentException: JWT secret must be at least 32 bytes`	Biến `NEXTCHAT_JWT_SECRET` trong `.env` ngắn hơn 32 bytes.	Đặt chuỗi khóa dài tối thiểu 32 ký tự ASCII (hoặc dùng `openssl rand -base64 48`), sau đó chạy lại `set -a && . ./.env && set +a`.
`IllegalStateException: NEXTCHAT_JWT_SECRET must`	Chạy ứng dụng ở profile khác `dev`/`test` nhưng vẫn dùng khóa	Khai báo `NEXTCHAT_JWT_SECRET` riêng biệt trong biến môi trường hệ

`be configured outside dev and test profiles`	secret mặc định.	thống trước khi chạy.
`Connection to localhost:5432 refused` hoặc lỗi xác thực `password authentication failed`	Container PostgreSQL chưa chạy hoặc khởi chạy `docker compose up -d` thiếu cờ `--env-file .env` làm lệch giá trị mặc định.	Chạy `cd nextchat && docker compose down -v && docker compose --env-file .env up -d`, đợi `healthy` rồi nạp lại `.env`.
`FlywayValidateException: Migration checksum mismatch`	Đã chỉnh sửa nội dung file `V1` hoặc `V2` sau khi Flyway đã áp dụng vào database.	Không sửa file migration đã chạy; hãy tạo file `V3__...sql` mới, hoặc xóa volume CSDL dev bằng `docker compose down -v` rồi tạo lại.
Giao diện báo `WS: Disconnected` hoặc nhận khung `ERROR` khi `SUBSCRIBE`	Access Token hết hạn, sai Origin hoặc tài khoản không phải thành viên của phòng chat.	Kiểm tra cột **Protocol Log** bên phải giao diện, đăng nhập lại để lấy token mới và đảm bảo truy cập đúng `http://localhost:8080`.
Testcontainers báo `Could not find a valid Docker environment` khi chạy `./mvnw clean verify`	Docker daemon chưa bật hoặc user hiện tại chưa có quyền truy cập `/var/run/docker.sock`.	Khởi động dịch vụ Docker (`sudo systemctl start docker`) và kiểm tra bằng lệnh `docker ps`.